

Arrays in C

Arrays in C are a kind of data structure that can store a fixed-size sequential collection of elements of the same **data type**. Arrays are used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.

What is an Array in C?

An array in C is a collection of data items of similar data type. One or more values same data type, which may be primary data types (int, float, char), or user-defined types such as struct or pointers can be stored in an array. In C, the type of elements in the array should match with the data type of the array itself.

The size of the array, also called the length of the array, must be specified in the declaration itself. Once declared, the size of a C array cannot be changed. When an array is declared, the compiler allocates a continuous block of memory required to store the declared number of elements.

Why Do We Use Arrays in C?

Arrays are used to store and manipulate the similar type of data.

Suppose we want to store the marks of 10 students and find the average. We declare 10 different variables to store 10 different values as follows –

```
int a = 50, b = 55, c = 67, . . . ;  
float avg = (float)(a + b + c + . . . ) / 10;
```

These variables will be scattered in the memory with no relation between them. Importantly, if we want to extend the problem of finding the average of 100 (or more) students, then it becomes impractical to declare so many individual variables.

Arrays offer a compact and memory-efficient solution. Since the elements in an array are stored in adjacent locations, we can easily access any element in relation to the current element. As each element has an index, it can be directly manipulated.

Example: Use of an Array in C

To go back to the problem of storing the marks of 10 students and find the average, the solution with the use of array would be –

[Open Compiler](#)

```
#include <stdio.h>

int main(){
    int marks[10] = {50, 55, 67, 73, 45, 21, 39, 70, 49, 51};
    int i, sum = 0;
    float avg;

    for (i = 0; i <= 9; i++){
        sum += marks[i];
    }

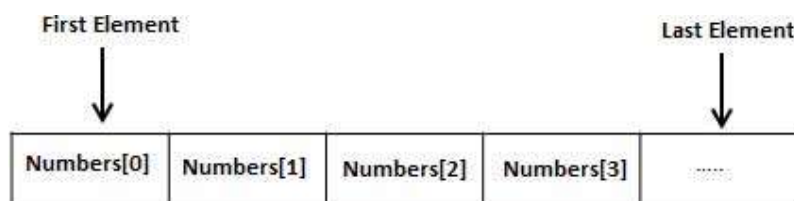
    avg = (float)sum / 10;
    printf("Average: %f", avg);
    return 0;
}
```

Output

Run the code and check its output –

Average: 52.000000

Array elements are stored in contiguous memory locations. Each element is identified by an index starting with "0". The lowest address corresponds to the first element and the highest address to the last element.



Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

Declaration of an Array in C

To declare an array in C, you need to specify the type of the elements and the number of elements to be stored in it.

Syntax to Declare an Array

```
type arrayName[size];
```

The "size" must be an integer constant greater than zero and its "type" can be any valid C data type. There are different ways in which an array is declared in C.

Example: Declaring an Array in C

In the following example, we are declaring an array of 5 integers and printing the indexes and values of all array elements –

[Open Compiler](#)

```
#include <stdio.h>

int main(){
    int arr[5];
    int i;

    for (i = 0; i <= 4; i++){
        printf("a[%d]: %d\n", i, arr[i]);
    }
    return 0;
}
```

Output

Run the code and check its output –

```
a[0]: -133071639
a[1]: 32767
a[2]: 100
a[3]: 0
a[4]: 4096
```

Initialization of an Array in C

At the time of declaring an array, you can initialize it by providing the set of comma-separated values enclosed within the curly braces {}.

Syntax to Initialize an Array

```
data_type array_name [size] = {value1, value2, value3, ...};
```

Example to Initialize an Array

The following example demonstrates the initialization of an integer array:

</>

Open Compiler

```
// Initialization of an integer array
#include <stdio.h>

int main()
{
    int numbers[5] = {10, 20, 30, 40, 50};

    int i; // loop counter

    // Printing array elements
    printf("The array elements are : ");
    for (i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }

    return 0;
}
```

Output

```
The array elements are : 10 20 30 40 50
```

Example of Initializing all Array Elements to 0

To initialize all elements to 0, put it inside curly brackets

</>

Open Compiler

```
#include <stdio.h>

int main(){
```

```

int arr[5] = {0};
int i;

for(i = 0; i <= 4; i++){
    printf("a[%d]: %d\n", i, arr[i]);
}
return 0;
}

```

Output

When you run this code, it will produce the following output –

```

a[0]: 0
a[1]: 0
a[2]: 0
a[3]: 0
a[4]: 0

```

Example of Partial Initialization of an Array

If the list of values is less than the size of the array, the rest of the elements are initialized with "0".


[Open Compiler](#)

```

#include <stdio.h>

int main(){
    int arr[5] = {1,2};
    int i;

    for(i = 0; i <= 4; i++){
        printf("a[%d]: %d\n", i, arr[i]);
    }
    return 0;
}

```

Output

When you run this code, it will produce the following output –

```
a[0]: 1
a[1]: 2
a[2]: 0
a[3]: 0
a[4]: 0
```

Example of Partial and Specific Elements Initialization

If an array is partially initialized, you can specify the element in the square brackets.

[Open Compiler](#)

```
#include <stdio.h>

int main(){
    int a[5] = {1,2, [4] = 4};
    int i;

    for(i = 0; i <= 4; i++){
        printf("a[%d]: %d\n", i, a[i]);
    }
    return 0;
}
```

Output

On execution, it will produce the following output –

```
a[0]: 1
a[1]: 2
a[2]: 0
a[3]: 0
a[4]: 4
```

Getting Size of an Array in C

The compiler allocates a continuous block of memory. The size of the allocated memory depends on the data type of the array.

Example 1: Size of Integer Array

If an integer array of 5 elements is declared, the array size in number of bytes would be "sizeof(int) x 5"

</>

Open Compiler

```
#include <stdio.h>

int main(){
    int arr[5] = {1, 2, 3, 4, 5};
    printf("Size of array: %ld", sizeof(arr));
    return 0;
}
```

Output

On execution, you will get the following output –

```
Size of array: 20
```

The **sizeof** operator returns the number of bytes occupied by the variable.

Example 2: Adjacent Address of Array Elements

The size of each **int** is 4 bytes. The compiler allocates adjacent locations to each element.

</>

Open Compiler

```
#include <stdio.h>

int main(){
    int a[] = {1, 2, 3, 4, 5};
    int i;

    for(i = 0; i < 4; i++){
        printf("a[%d]: %d \t Address: %d\n", i, a[i], &a[i]);
    }
    return 0;
}
```

Output

Run the code and check its output –

```
a[0]: 1      Address: 2102703872
a[1]: 2      Address: 2102703876
a[2]: 3      Address: 2102703880
a[3]: 4      Address: 2102703884
```

In this array, each element is of **int** type. Hence, the 0th element occupies the first 4 bytes 642016 to 19. The element at the next subscript occupies the next 4 bytes and so on.

Example 3: Array of Double Type

If we have the array type of **double** type, then the element at each subscript occupies 8 bytes


[Open Compiler](#)

```
#include <stdio.h>

int main(){
    double a[] = {1.1, 2.2, 3.3, 4.4, 5.5};
    int i;

    for(i = 0; i < 4; i++){
        printf("a[%d]: %f \t Address: %ld\n", i, a[i], &a[i]);
    }
    return 0;
}
```

Output

Run the code and check its output –

```
a[0]: 1.100000 Address: 140720746288624
a[1]: 2.200000 Address: 140720746288632
a[2]: 3.300000 Address: 140720746288640
a[3]: 4.400000 Address: 140720746288648
```

Example 4: Size of Character Array

The length of a "char" variable is 1 byte. Hence, a char array length will be equal to the array size.

</>

Open Compiler

```
#include <stdio.h>

int main(){
    char a[] = "Hello";
    int i;

    for (i=0; i<5; i++){
        printf("a[%d]: %c address: %ld\n", i, a[i], &a[i]);
    }
    return 0;
}
```

Output

Run the code and check its output –

```
a[0]: H address: 6422038
a[1]: e address: 6422039
a[2]: l address: 6422040
a[3]: l address: 6422041
a[4]: o address: 6422042
```

Accessing Array Elements in C

Each element in an array is identified by a unique incrementing index, starting with "0". To access the element by its index, this is done by placing the index of the element within square brackets after the name of the array.

The elements of an array are accessed by specifying the index (offset) of the desired element within the square brackets after the array name. For example –

```
double salary = balance[9];
```

The above statement will take the 10th element from the array and assign the value to the "salary".

Example to Access Array Elements in C

The following example shows how to use all the three above-mentioned concepts viz. declaration, assignment, and accessing arrays.

[Open Compiler](#)

```
#include <stdio.h>

int main(){

    int n[5]; /* n is an array of 5 integers */
    int i, j;

    /* initialize elements of array n to 0 */
    for(i = 0; i < 5; i++){
        n[i] = i + 100;
    }

    /* output each array element's value */
    for(j = 0; j < 5; j++){
        printf("n[%d] = %d\n", j, n[j]);
    }
    return 0;
}
```

Output

On running this code, you will get the following output –

```
n[0] = 100
n[1] = 101
n[2] = 102
n[3] = 103
n[4] = 104
```

The index gives random access to the array elements. An array may consist of struct variables, pointers and even other arrays as its elements.

More on C Arrays

C - Properties of Array

Arrays are a very important data structure in C. Use of array in C program makes it easier to handle large amount of data. Arrays have a number of advantages over singular variables as a result of their number of properties. Most of the important properties of array are a result of its composition – that an array is a collection of values of same **data type**, and in a continuous block of memory.

Array properties may change in the different programming languages. In C programming language, the following are the main properties of arrays:

- Collection of Same Data Type
- Contiguous Memory Allocation
- Fixed Size
- Length Depends on Type
- Indexing
- Pointer Relationship
- Lower and Upper Bounds
- Multi-dimensional Array
- Implementation of Complex Data Structures

Let us discuss each property in detail.

Collection of Same Data Type

All elements of an **array** must be of the same data type. This ensures consistent access and operations on the data.

If an array is declared as follows –

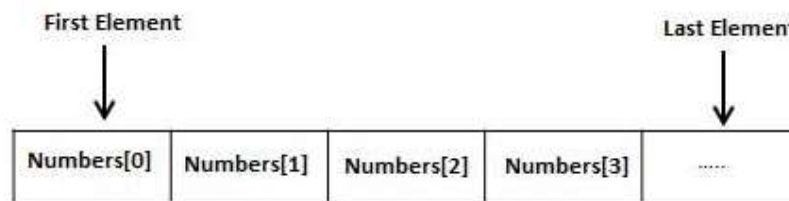
```
int arr[] = {50, 67.55, "hello", 21};
```

Compiler issues a warning –

```
initialization of 'int' from 'char *' makes integer from pointer without a cast  
[-Wint-conversion]
```

Contiguous Memory Allocation

All elements of an array are stored in contiguous memory locations, meaning they occupy a block of memory next to each other. This allows for efficient random access and memory management.



Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

Fixed Size

The size of an array is fixed at the time of declaration and cannot be changed during the program's execution. This means you need to know the maximum number of elements you need beforehand. In C, an array cannot have a size defined in terms of a variable.

//This is accepted

```
#define SIZE = 10
int arr[SIZE];
```

//This is also accepted

```
const SIZE = 10;
int arr[SIZE];
```

//This is not accepted

```
int SIZE = 10;
int arr[SIZE];
```

//The size must be an integer. This will give error

```
float num[10.5] = {50, 55, 67, 73, 45, 21, 39, 70, 49, 51};
```

Length Depends on Type

Since an array can store all the elements of same type, the total memory occupied by it depends on the data type.

Example


[Open Compiler](#)

```
#include<stdio.h>

int main() {
    int num[10] = {50, 55, 67, 73, 45, 21, 39, 70, 49, 51};
    int size = sizeof(num) / sizeof(int);
    printf("element at lower bound num[0]: %d \n", num[0]);
    printf("at upper bound: %d byte \n", num[size-1]);
    printf("length of int array: %ld \n", sizeof(num));
    double nm[10] = {50, 55, 67, 73, 45, 21, 39, 70, 49, 51};
    size = sizeof(nm) / sizeof(double);
    printf("element at lower bound nm[0]: %f \n", nm[0]);
    printf("element at upper bound: %f \n", nm[size-1]);
    printf("byte length of double array: %ld \n", sizeof(nm));

    return 0;
}
```

Output

```
element at lower bound num[0]: 50
at upper bound: 51 byte
length of int array: 40
element at lower bound nm[0]: 50.000000
element at upper bound: 51.000000
byte length of double array: 80
```

Indexing

Each element in an array has a unique index, starting from 0. You can access individual elements using their index within square brackets. Usually, array is traversed with a **for loop** running over its length and using the loop variable as the index.

Example

```
#include <stdio.h>
```

```
int main() {
    int a[] = {1,2,3,4,5};
    int i;

    for (i=0; i<4; i++){
        printf("a[%d]: %d \n", i, a[i]);
    }
    return 0;
}
```

Pointer Relationship

The name of an array is equivalent to a constant **pointer** to its first element. This lets you use array names and pointers interchangeably in certain contexts.

Example

</>

Open Compiler

```
#include <stdio.h>
int main() {
    int num[10] = {50, 55, 67, 73, 45, 21, 39, 70, 49, 51};
    printf("num[0]: %d Address of 0th element: %d\n", num[0], &num[0]);
    printf("Address of array: %d", num);
    return 0;
}
```

Output

```
num[0]: 50 Address of 0th element: 6422000
Address of array: 6422000
```

Lower and Upper Bounds

Each element in an array is identified by an index starting with 0. The lower bound of an array is the index of its first element, which is always 0. The last element in the array is identified by the array size -1 as its index.

Example



Open Compiler

```
#include <stdio.h>
int main() {
    int num[10] = {50, 55, 67, 73, 45, 21, 39, 70, 49, 51};
    int size = sizeof(num) / sizeof(int);
    printf("element at lower bound num[0]: %d at upper bound: %d Size of array: %d",
        num[0], num[size-1], size);
    return 0;
}
```

Output

```
element at lower bound num[0]: 50 at upper bound: 51 Size of array: 10
```

Multi-dimensional Array

The array is declared with one value of size in square brackets , it is called one dimensional array. In a one dimensional array, each element is identified by its index or subscript. In C, you can declare with more indices to simulate a two, three or **multidimensional array**.

Example

For example, following is the example of a two-dimensional array –

```
int a[3][3] = { {1, 2, 3}, {11, 22, 33}, {111, 222, 333}};
```

You can think of a one dimensional array as a list, and a two dimensional array as a table or a matrix. Theoretically, there is no limit to the number of dimensions of an array, but in practice, two-dimensional arrays are used in design of spreadsheets, databases etc.

Implementation of Complex Data Structures

We can use an array in the construction of a struct data type to implement data structures such as stack, linked list and trees.

Example

```
typedef struct stack {  
    int top;  
    int arr[10];  
} Stack;
```

Thus, array is an important tool in the armoury of the programmer, as it can be used for different applications. The concept of array in C is implemented by many subsequent programming languages such as C++, C#, Java etc.